

INFORME DE CLASE numero 0003: PROGRAMACIÓN ORIENTADA A OBJETOS EN JAVA

**Tema: Clases Abstractas, Herencia y
Polimorfismo**

Nombre: Milagros Belen Martinez Sonco

Profesor: Jose Sumire Diaz

Fecha: 10/09/2026

1. OBJETIVO

*Implementar un sistema de recompensas para una tienda usando los 3 pilares de POO:
Abstracción, Herencia y Polimorfismo.*

2. DESARROLLO DEL TEMA

a. CLASE MADRE ABSTRACTA: `ItemRecompensa`

Es el "molde" general. No se puede crear un objeto directamente de ella.

```
public abstract class ItemRecompensa{  
  
    private String nombre;  
  
    private int costoPuntos;  
  
  
    public ItemRecompensa(String nombre, int costoPuntos) { ... }  
}
```

```
public abstract void aplicarEfecto(); // Método obligatorio para las hijas
```

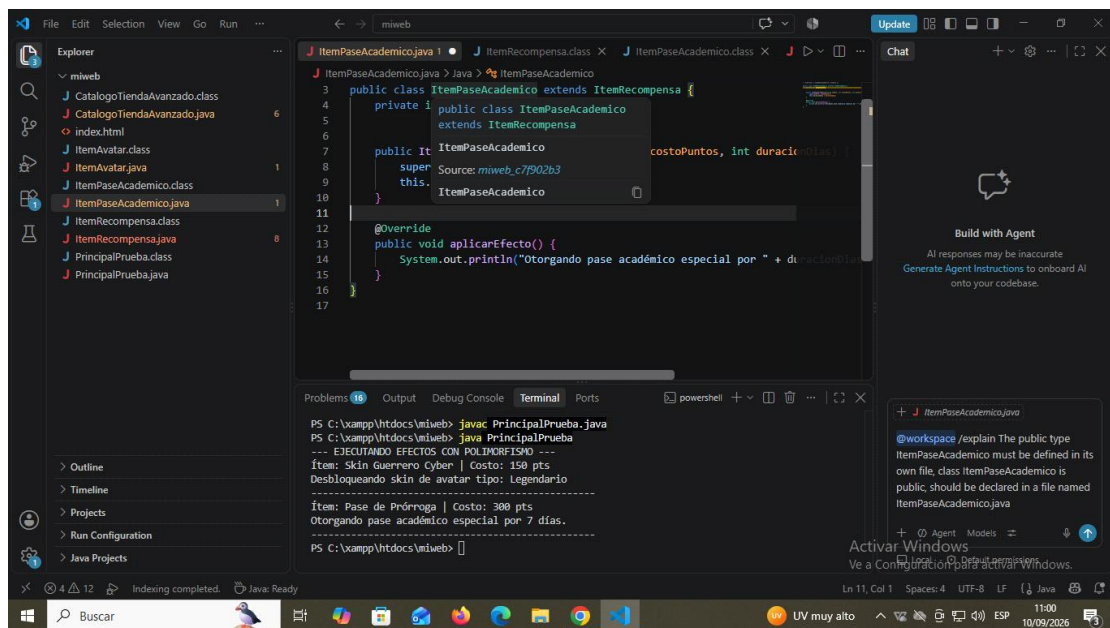
```
public String getNombre() { return nombre; }
```

```
public int getCostoPuntos() { return costoPuntos; }
```

```
}
```

Características:

1. ``abstract``: No se puede instanciar. ``new ItemRecompensa()`` da error.
2. ``aplicarEfecto()``: Método abstracto. Obliga a todas las subclases a implementarlo.
3. Guarda los atributos comunes: todo item tiene ``nombre`` y ``costoPuntos``.



B. SUBCLASE 1: ``ItemAvatar``

Representa una skin para el avatar del usuario.

```
public class ItemAvatar extends ItemRecompensa {
```

```
    private String tipoSkin; // Atributo propio
```

```

public ItemAvatar(String nombre, int costoPuntos, String tipoSkin) {

    super(nombre, costoPuntos); // Llama al constructor del padre

    this.tipoSkin = tipoSkin;

}

```

@Override

```

public void aplicarEfecto() {

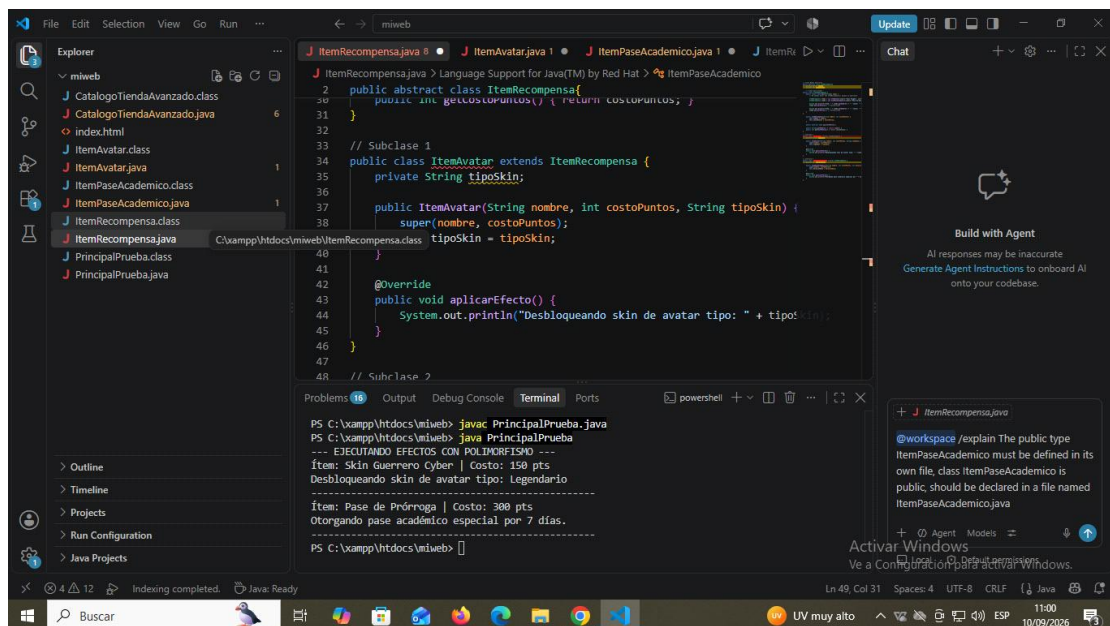
    System.out.println("Desbloqueando skin de avatar tipo: " + tipoSkin);

}

}

```

Qué aporta?: El atributo `tipoSkin` y la forma específica de aplicar el efecto.



C. SUBCLASE 2: `ItemPaseAcademico`

Representa un pase que da beneficios por cierta cantidad de días.

```
public class ItemPaseAcademico extends ItemRecompensa {  
  
    private int duracionDias; // Atributo propio  
  
    public ItemPaseAcademico(String nombre, int costoPuntos, int duracionDias) {  
  
        super(nombre, costoPuntos);  
  
        this.duracionDias = duracionDias;  
  
    }  
  
    @Override  
  
    public void aplicarEfecto() {  
  
        System.out.println("Otorgando pase académico especial por " + duracionDias  
+ " días.");  
  
    }  
  
}
```

**Qué aporta:* El atributo `duracionDias` y su propio `aplicarEfecto()`.*

3. CONCEPTOS CLAVE APLICADOS

Concepto , Ejemplo en el código, Para qué sirve

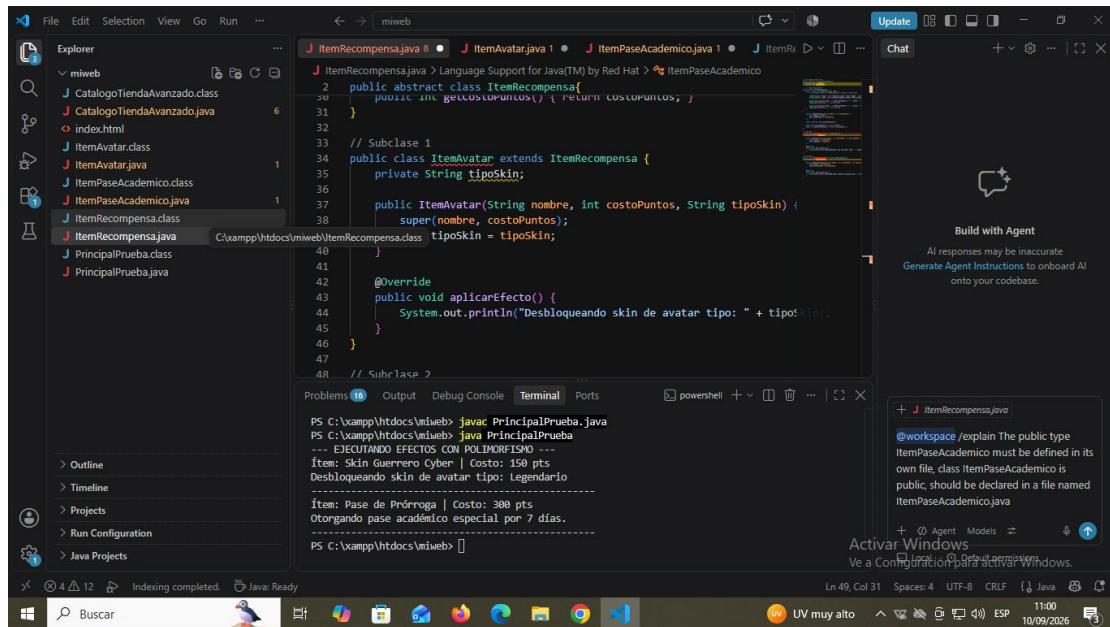
Abstracción , `public abstract class ItemRecompensa` , Ocultar lo general y obligar a definir lo específico

Herencia , `extends ItemRecompensa` , Reutilizar código. Las hijas heredan `nombre` y `costoPuntos`

Polimorfismo , `ItemRecompensa item = new ItemAvatar(...)`

`item.aplicarEfecto()` , Tratar objetos distintos como si fueran del mismo tipo. Java decide qué método ejecutar

Sobreescritura , `@Override public void aplicarEfecto()` , Cada hija da su propia versión del método abstracto



4. CLASE DE PRUEBA: `Main`

Para probar el polimorfismo guardamos objetos distintos en un arreglo del tipo padre.

public class Main {

public static void main(String[] args) {

ItemRecompensa skin = new ItemAvatar("Skin Ninja", 300, "Ninja Sombras");

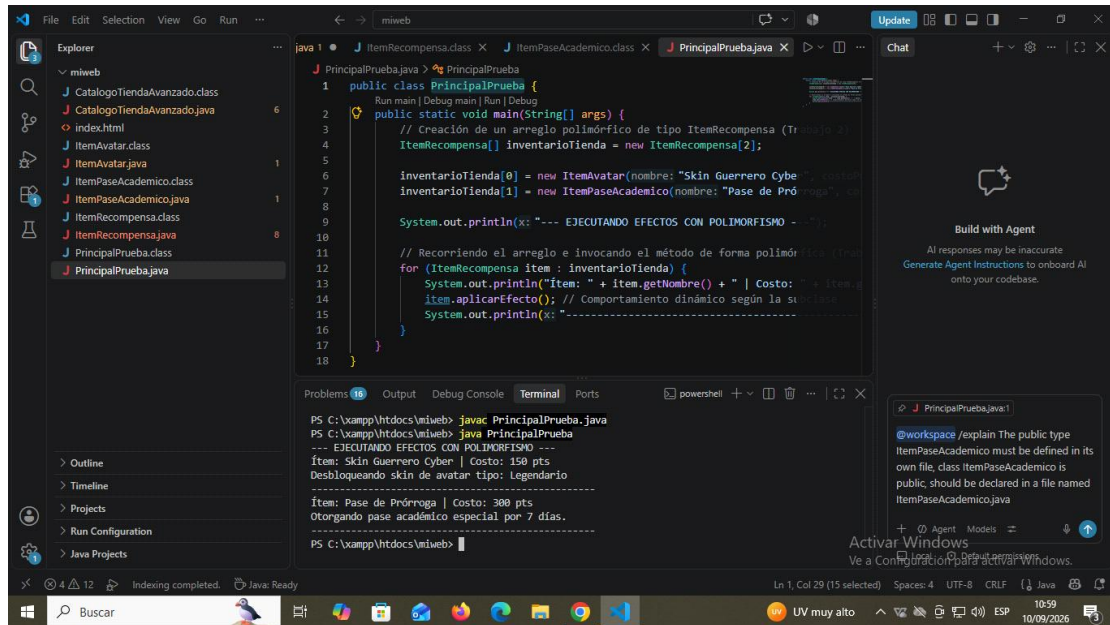
ItemRecompensa pase = new ItemPaseAcademico("Pase Finales", 1000, 14);

skin.aplicarEfecto(); // Ejecuta el de ItemAvatar

pase.aplicarEfecto(); // Ejecuta el de ItemPaseAcademico

}

}



Salida:

Desbloqueando skin de avatar tipo: Ninja Sombras

Otorgando pase académico especial por 14 días.

5. CONCLUSIÓN

Hoy implementamos un sistema extensible. Si mañana queremos agregar `ItemMonedas` o `ItemFondo`, solo creamos una nueva clase que `extienda` `ItemRecompensa` e implementa `aplicarEfecto()`. El resto del código no se toca.

Ventaja principal: Gracias a la clase abstracta + polimorfismo, la "Tienda" puede manejar cualquier tipo de recompensa sin saber qué tipo exacto es.